

Парадигма выполнения расчётов в Mozart ERP

Основные принципы

1. Все расчёты — экземпляры сущностей класса CALC

- Хранятся в `data.entity_instances + data.entity_versions + ext_calc`
- Имеют версионирование, права доступа, историю изменений
- Могут быть включены/выключены, иметь статус (DEV/PROD)

2. Единый объект выполнения расчётов

- В L3 (клиент) — `app.ORunCalc()`
- В L2 (сервер) — `L2_CalcManager`
- В L3 и L2 есть ровно один объект, который умеет вызывать расчёты

3. Расчёт может выполняться на любом слое

- На L3 — быстрые локальные расчёты (валидация форм, простые формулы)
- На L2 — основные бизнес-расчёты (зарплата, налоги, амортизация)
- На L2 в изолированном ядре — тяжёлые расчёты (отчёты за год)

4. Единый мост L3 ↔ L2

- L3 не знает, как отправлять запросы на L2. Это делает один объект `L3_DataBridge`
- L2 не знает, откуда пришли запросы. Их принимает один объект `L2_DataBridge`
- Все расчёты, выполняемые на L2, идут через этот мост

5. Безопасность через стандартную систему прав

- Запуск расчёта = действие `EXECUTE` над экземпляром `CALC`
- Права на расчёт наследуются от роли пользователя через `auth.permissions`
- Переданные параметры тоже проверяются (что можно передавать, а что нельзя)

Жизненный цикл расчёта

1. Регистрация расчёта (в конфигураторе)

- Создаётся сущность класса `CALC` (экземпляр + версия)
- В `ext_calc` указываются:

- `cfilepath` — путь к исполняемому файлу (на L2)
- `calias` — системный алиас (уникальный)
- `cname` — человеческое название
- `mcomment` — описание
- `lisactive` — активен/неактивен
- `xparams_json` — схема входных/выходных параметров (JSON Schema)
- `chash` — хэш кода (для контроля целостности)

2. Вызов расчёта (из L3)

python

```
result = app.ORunCalc(
    calc_alias="payroll_calculation", # алиас расчёта
    level=Level.L2,                  # где выполнять: L2 / L3
    async_mode=False,                # синхронно / асинхронно
    kernel=Kernel.ISOLATE,           # как выполнять: main / thread /
isolate
    params={                          # входные параметры (значения)
        "period": "2025-01",
        "department_id": 123
    },
    ref_params={                     # ссылочные параметры (указатели)
        "document": "invoice:456"    # форма/документ, откуда брать данные
    },
    output_params=["result", "errors"], # какие параметры вернуть
    callback=on_calc_finished         # для async_mode=True
)
```

3. Маршрутизация запроса

- `app.ORunCalc` проверяет `level`
- Если `level = L3` → передаёт в `L3_CalcManager` (локальное выполнение)
- Если `level = L2` → передаёт в `L3_DataBridge`

`L3_DataBridge` формирует пакет:

json

```
{
    "command": "run_calculation",
    "calc_alias": "payroll_calculation",
    "async": false,
    "params": {...},
    "ref_params": {...},
    "output_params": ["result", "errors"],
    "session_id": "uuid"
}
```

и отправляет на L2.

4. Приём и выполнение на L2

- L2_DataBridge получает пакет, проверяет авторизацию
- Передаёт в L2_CalcManager
- L2_CalcManager:
 - Находит экземпляр CALC по алиасу
 - Проверяет права пользователя (EXECUTE)
 - Проверяет, активен ли расчёт (lisactive = TRUE)
 - Проверяет параметры по схеме (xparams_json)
 - Загружает код из cfilepath
 - Запускает выполнение (в потоке, процессе или синхронно)
 - Собирает результат

5. Возврат результата

- Результат идёт обратным путём: L2_CalcManager → L2_DataBridge → L3_DataBridge → app.ORunCalc (или callback)

6. Асинхронный режим

- При async_mode=True L3 не ждёт результат
- Результат приходит через callback (или событие)
- Можно отслеживать статус выполнения (running/finished/error)

Права доступа к расчётам

Действие	Проверка
Выполнить расчёт	EXECUTE на экземпляре CALC
Создать расчёт	CREATE на entitytype CALC
Редактировать расчёт	UPDATE на экземпляре CALC
Удалить расчёт	DELETE на экземпляре CALC
Смотреть код расчёта	READ на экземпляре CALC
Наследование прав: как для любой сущности в Mozart ERP	

Где живут расчёты

Тип расчёта	Где выполняется	Пример
Валидация формы	L3	Проверка заполнения полей
Простая формула	L3	Сумма строки документа
Справочные данные	L2 (быстро)	Загрузка списка валют
Бизнес-расчёт	L2 (thread)	Расчёт амортизации, зарплат
Тяжёлый отчёт	L2 (isolate)	Годовой баланс

Ночная обработка L2 (isolate, async) Закрытие месяца

Безопасность передачи параметров

Проблема: Передаём ссылку на документ "invoice:456" → что, если пользователь передаст документ, на который у него нет прав?

Решение:

- L2 при получении `ref_params` проверяет каждый указатель
- Проверяет права пользователя на `READ` этого документа
- Если прав нет — расчёт не запускается, ошибка

Выходные параметры как указатели (`output_params`) позволяют расчёту вернуть результат прямо в форму (например, заполнить строки документа). Это удобно, но должно быть строго контролируемо.

Оценка сложности схемы

Что сложно:

1. **Маршрутизация** — нужно поддерживать два режима (L2/L3) и два варианта ядра (sync/async, thread/isolate)
2. **Асинхронность** — управление колбэками, отслеживание статуса
3. **Безопасность указателей** — проверка прав на каждый переданный объект
4. **Версионирование расчётов** — старый код должен выполняться для старых периодов

Что упрощает:

1. **Единый мост L3 ↔ L2** — только один объект знает, как слать запросы
2. **Единый менеджер расчётов на L2** — вся логика выполнения в одном месте
3. **Стандартные права доступа** — не нужно придумывать новую модель безопасности
4. **CALC как обычная сущность** — работает единая схема версионирования и прав

Упрощения, которые можно сделать (но не сейчас)

Сложность	Упрощение	Цена
Выполнение на L3	Запретить, всегда гонять на L2	Потеря производительности для простых расчётов
Асинхронный режим	Только синхронный	Придется ждать, пока расчёт завершится
Разные ядра (thread/isolate)	Только thread	Риск зависания L2 при тяжёлых расчётах
Указатели на объекты	Только передача по значению	Нельзя вернуть результат прямо в форму
Пока оставляем всё как есть. Упрощать будем, если что-то реально мешает.		

Почему эта схема правильная

1. **Безопасность** — встроена через стандартные права `auth.permissions`
2. **Масштабируемость** — расчёты можно раскидать по разным ядрам/машинам
3. **Прозрачность** — версионирование, хэши, аудит
4. **Гибкость** — расчёт может выполняться где угодно (L2/L3, thread/isolate)
5. **Единообразие** — один способ вызова для всех расчётов

Оценка: Схема **сложная, но правильная**. Она даёт максимум контроля и безопасности. Со временем, если какая-то часть окажется не нужна — её можно отключить или упростить. Но база (единый мост, единый вызов, права доступа) — остаётся.

Итог: Оставляем как есть. Это тот случай, когда сложность оправдана.

Гибридное хранение кода в CALC-сущности

Поля `ext_calc` (таблица расширения для сущности CALC)

Поле	Тип	Назначение
<code>calias</code>	VARCHAR(100)	Системный алиас
<code>sname</code>	VARCHAR(500)	Человеческое название
<code>ccode</code>	TEXT	Код в базе (если заполнено — используем его)
<code>cfilepath</code>	TEXT	Путь к файлу (если <code>ccode</code> пуст)
<code>chash</code>	VARCHAR(64)	Хэш кода (для контроля целостности)
<code>mcomment</code>	TEXT	Описание
<code>lisactive</code>	BOOLEAN	Активен

Правило: Если `ccode` не пуст — используем код из БД. Если пуст — считываем из файла по `cfilepath`.

Где что храним

Тип кода	Хранение	Почему
Логика форм	В БД (<code>ccode</code>)	Часто меняется, нужен быстрый доступ, версионирование, не требует внешней IDE
Расчёт зарплаты	В файле (<code>cfilepath</code>)	Большой объём, редкие изменения, удобно править в IDE, можно хранить в Git
Расчёт амортизации	В файле	Аналогично
Налоговый расчёт	В файле	Аналогично
Маленькие вспомогательные функции	В БД	Быстро, не плодим файлы

Логика загрузки кода в L2/L3

python

```
def get_calc_code(calc_alias):
    """Возвращает код CALC-сущности"""
    calc = load_calc_instance(calc_alias)

    if calc.ccode and calc.ccode.strip():
        # Код в БД — используем его
        return calc.ccode
    elif calc.cfilepath and os.path.exists(calc.cfilepath):
        # Код в файле — читаем
        with open(calc.cfilepath, 'r', encoding='utf-8') as f:
            return f.read()
    else:
        raise Exception(f"Нет кода для расчёта {calc_alias}")
```

Для форм — только БД

При создании новой формы в конфигураторе:

1. Создаётся запись в `meta.forms`
 2. **Автоматически** создаётся CALC-сущность:
 - `calias = form_<form_alias>_logic`
 - `cname = Логика формы <form_alias>`
 - `ccode = шаблон кода` (пустая заглушка с комментариями)
 - `cfilepath = NULL`
 3. Всё редактирование кода — через встроенный редактор, который пишет прямо в `ccode`.
-

Для расчётов — выбор при создании

В конфигураторе при создании CALC-сущности разработчик выбирает:

text

- Хранить код в базе
- Хранить код в файле: [_____] (кнопка "Обзор")

Если выбран файл — `ccode` остаётся пустым, используется `cfilepath`.

Преимущества

Аспект

Преимущество

Формы	Код всегда под рукой, не нужно таскать файлы, версионирование через БД
Большие расчёты	Можно править в любимой IDE, хранить в Git, подключать отладчик
Гибкость	В любой момент можно перенести код из БД в файл и наоборот
Аудит	Хэш кода всегда актуален, независимо от места хранения
Бэкап	Бэкап БД сохраняет всё, включая код форм. Для расчётов — отдельный бэкап файлов (но можно и в БД)

Итог

Овцы целы, волки сыты.

- **Формы** — код в БД.
- **Расчёты (зарплата, налоги, амортизация)** — код в файлах, но при желании можно и в БД.

Всё через одну CALC-сущность и два поля: ccode и cfilepath.